

| Front-end



Contents

- 01.** React.js 개념과 프로젝트 생성
- 02.** Props & state, ref 의 개념과 Component
- 03.** SPA 앱 구현과 Router
- 04.** Next.js 의 개념과 Routing, 이미지 최적화
- 05.** Network 기능 활용과 로그인 인증 처리
- 06.** 라이브러리 활용과 데이터 공유, 빌드

과정 OVERVIEW

React.js

React.js 개념과 Virtual DOM

React 기본 개념 (state, props, component)

Data 다루기

Todo 리스트 만들어보기

Router

React.js
기본 개념

Data 를 활용한
U.I. 구현

React.js를 이용해
CSR 앱을 구현한다.

과정 OVERVIEW

Next.js

React.js와 Next.js의 차이

Next.js Routing

axios를 활용한 서버와의 통신

Session 처리

프로젝트에서 자주 사용되는 라이브러리

상태 공유와 Context & Redux

Next.js 기초
및 서버 통신

Project 를 위한
실전 개념

Next.js와 외부 라이브러리를
활용한 SSR 앱을 구현한다.

일정(일자별)

React.js 활용부터 Next.js를 활용한 서비스 제작까지

1일차	2일차	3일차
React.js의 개념과 활용	Next.js의 개념과 활용	Next.js의 프로젝트에서 활용
React.js 개념과 Virtual DOM	React.js와 Next.js의 개념 비교	로그인 인증과 JWT 활용
React 기본 개념 이해	Routing 활용	프로젝트에서 사용되는 주요 라이브러리
Data 활용과 앱 만들기	Next.js의 유용한 기능 활용	Prop drilling과 상태 공유의 정의
Router의 개념과 활용	서버와의 통신을 통한 앱 만들기	앱 제작과 배포

강의 목표

Next.js를 활용한 Server와 통신 가능한 서비스 제작

React.js의 개념과 활용

React.js의 기본 개념과 활용 방법에 대해서 알아 본다.

Next.js의 개념과 활용

React.js와 Next.js의 차이점을 확인해 보고 활용 방법에 대해서 알아 본다.

Next.js를 활용한 서비스 제작

Next.js를 활용하여 Server와 통신 가능한 서비스 제작을 해본다.

04.

Next.js의 개념과 Routing, 이미지 최적화

전체 흐름

Next.js 의 개념과 활용

React.js와 Next.js
의 차이

Server와의
통신을 위한
방법

서비스 제작을
위한 주요 기술들

Next.js

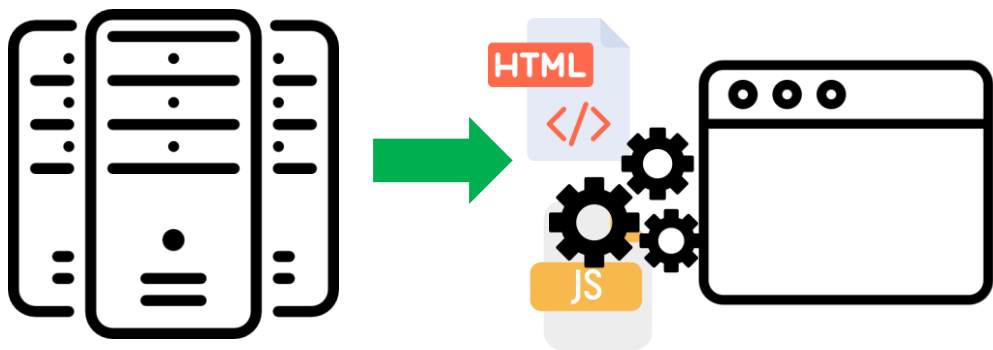
React.js의 단점

- React.js는 본래 SPA(Single Page App)를 위해 탄생한 웹 프레임워크
 - ➡ SPA에 특화된 프레임워크이지만 일반적으로 SPA에 사용하지 않음
- CSR(Client Side Rendering)을 활용해 페이지를 구성
 - ➡ 검색엔진에서 crawling을 제대로 할 수 없어 검색 노출에 불리함

Next.js

☑ React.js의 단점

- CSR : 서버에서 내려보내준 HTML과 JS를 이용해 client(브라우저)에서 화면을 그리는 방식
- React.js의 index.html을 확인해 보자



```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx">
  </script>
</body>
```

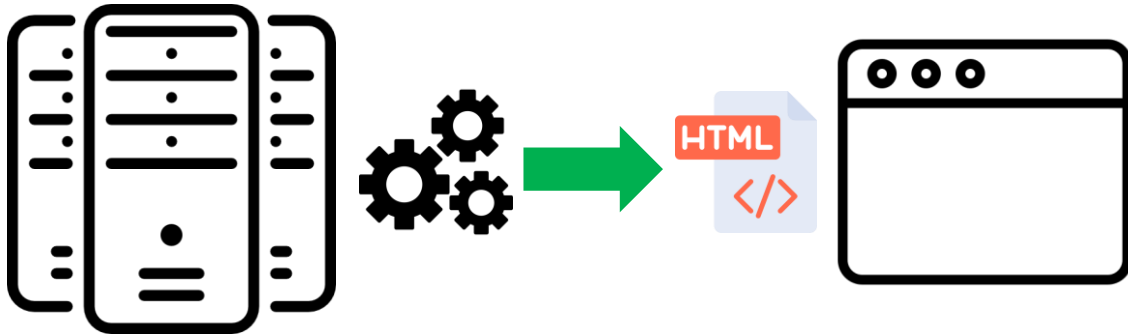
Next.js

☑ Next.js 란?

- Next.js는 React.js에서의 단점을 개선한 react.js의 framework

➡ Routing 기능을 React.js보다 쉽게 사용할 수 있다.

➡ SSR(Server Side Rendering)을 사용한다.

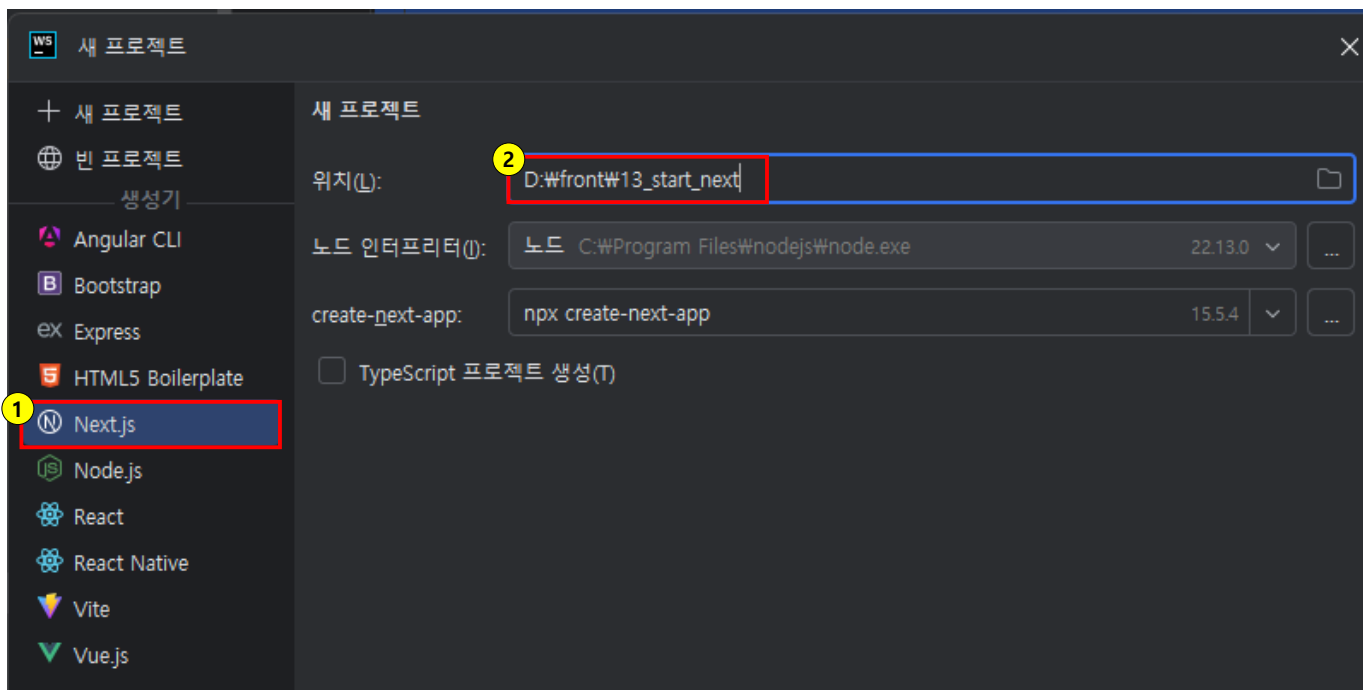


* Server Side Rendering은 서버에서 페이지를 완성해 Client로 내려주는 방식

Next.js 프로젝트 시작

✓ Next.js 시작

- Next.js 프로젝트를 생성해 내부 구조를 살펴보자
- Alt+₩ > File > New > Project... 선택 후 아래 내용대로 프로젝트를 생성해 보자



- ① Next.js 선택 (개발 및 빌드 도구)
- ② Project 생성 경로와 이름 지정 (5자 이상, 대문자X)

Next.js 프로젝트 시작

Next.js 시작

- 기본적 추천 세팅 또는 **직접 선택** *추천 세팅은 TypeScript 사용해야함

```
Yes, use recommended defaults
No, reuse previous settings
No, customize settings - Choose your own preferences
```



Next.js 프로젝트 시작

Next.js 시작

- 각 질문 항목에 대해서 지정해 주자

```
✓ Would you like to use TypeScript? ... No / Yes
✓ Which linter would you like to use? » None
✓ Would you like to use React Compiler? ... No / Yes
✓ Would you like to use Tailwind CSS? ... No / Yes
✓ Would you like your code inside a `src/` directory? ... No / Yes
✓ Would you like to use App Router? (recommended) ... No / Yes
✓ Would you like to customize the import alias (`@/*` by default)? ... No / Yes
```

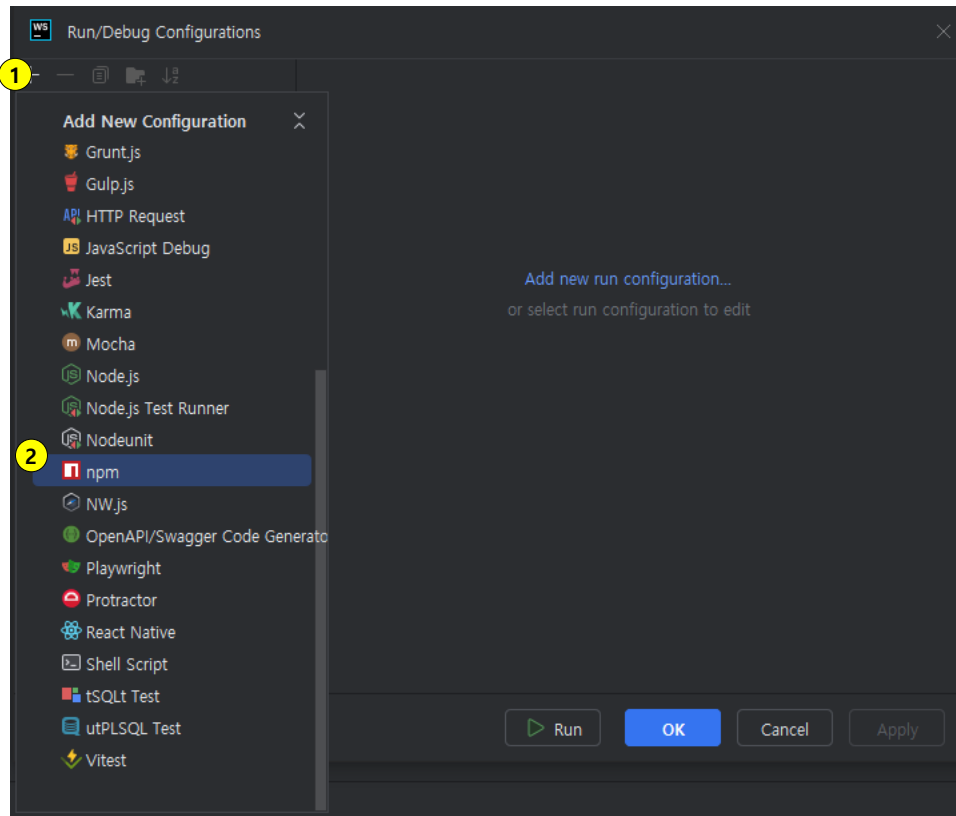
1	TypeScript를 사용 여부
2	Code linter 사용 여부 - 문법, 스타일 오류 자동 검사 도구
3	React.js 를 효율적으로 렌더링 할 수 있게 해주는 컴파일러(아직 안정화 버전 아님)
4	Tailwind CSS를 사용 여부 - 스타일링 프레임워크
5	소스 저장 디렉토리 - src 에 넣을지 root 에 바로 넣을지 여부
6	App Router를 사용 여부(사용권장) - webpack 이후 차세대 번들러
7	import 경로 별칭(alias) 커스터마이징 여부(기본값은 @/* → 예: import Button from "@components/Button")

Next.js 프로젝트 시작

☑ Next.js 시작

- 상단 실행 버튼이 활성화되지 않았다면 아래 항목
- Current File > Edit Configurations

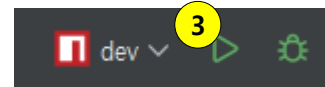
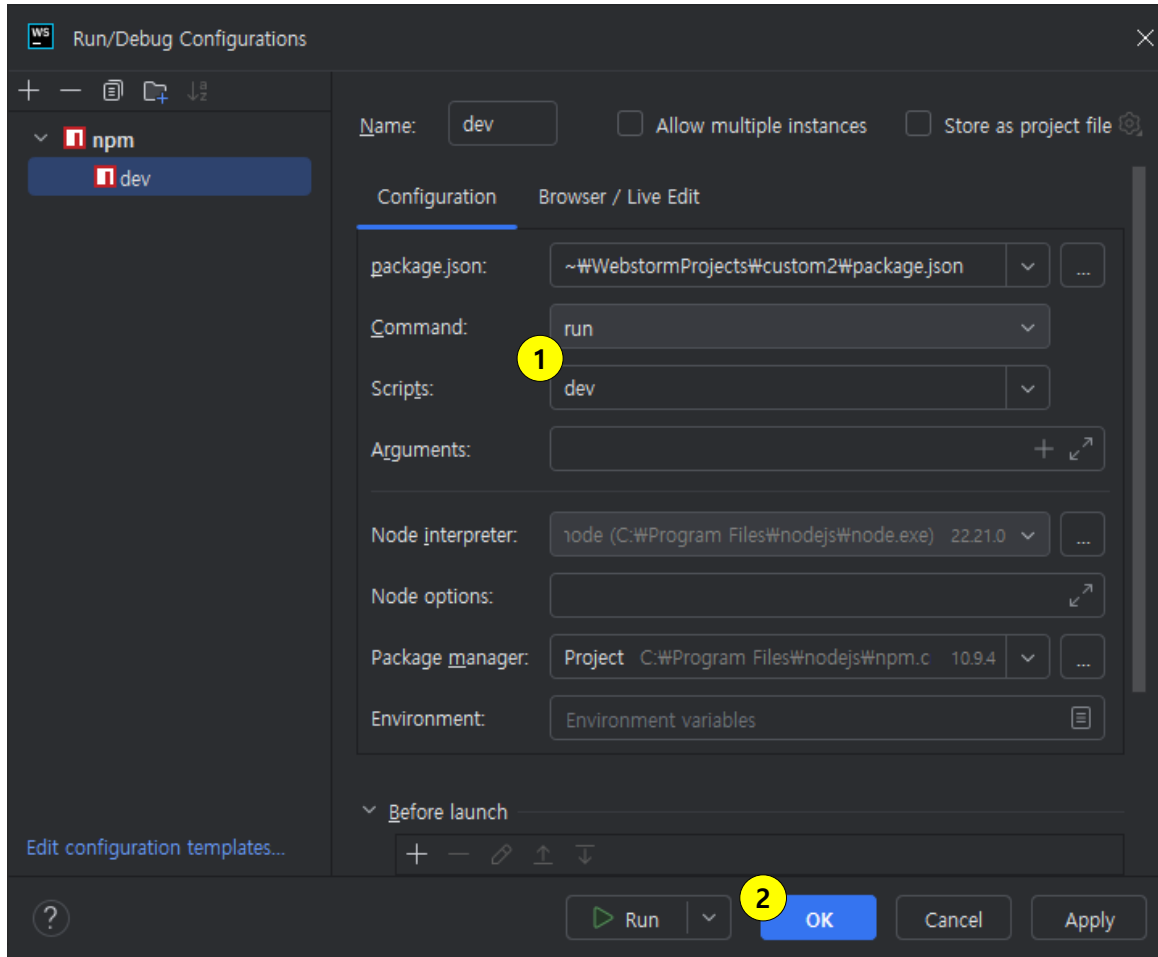
Current File ▾ ▶



- ① + 버튼을 눌러 설정 항목 pop up
- ② npm 선택

Next.js 프로젝트 시작

✓ Next.js 시작

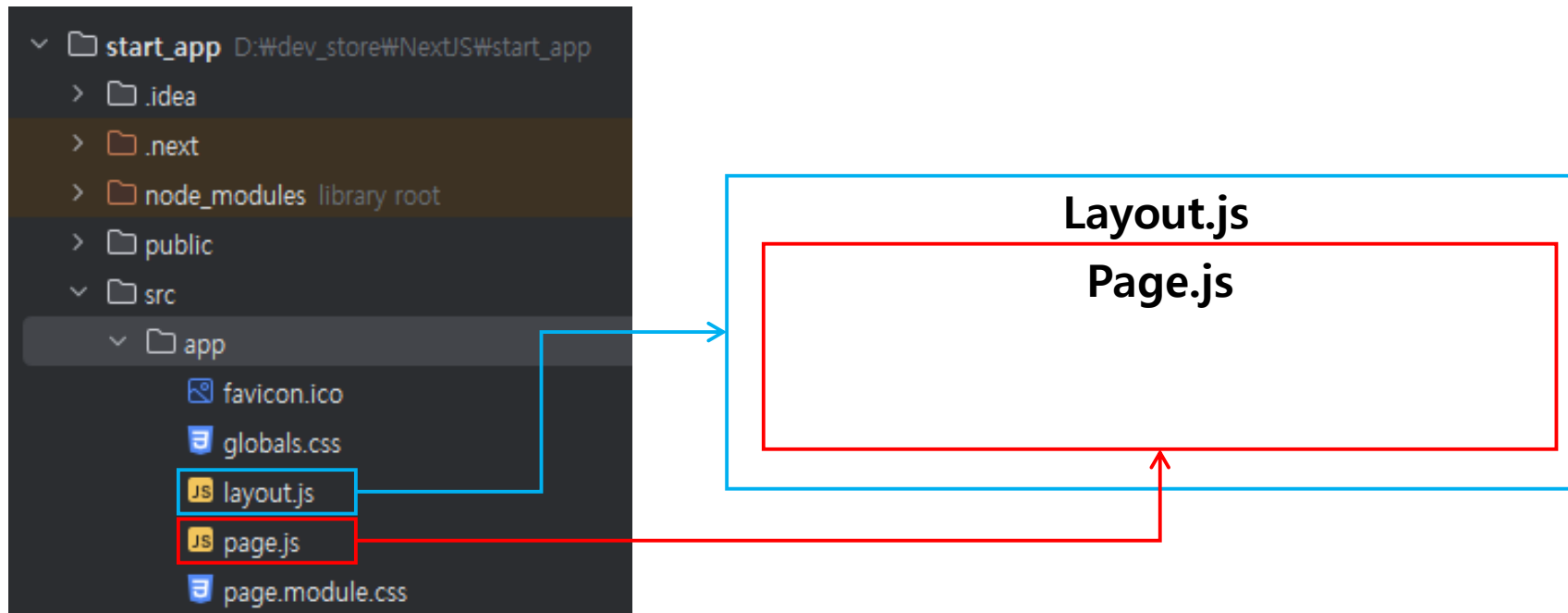


- ① Scripts 항목에서 dev 선택
- ② OK 버튼 클릭
- ③ 이후 상단에 play 버튼 활성화

Next.js 프로젝트 시작

☑ Next.js 구조

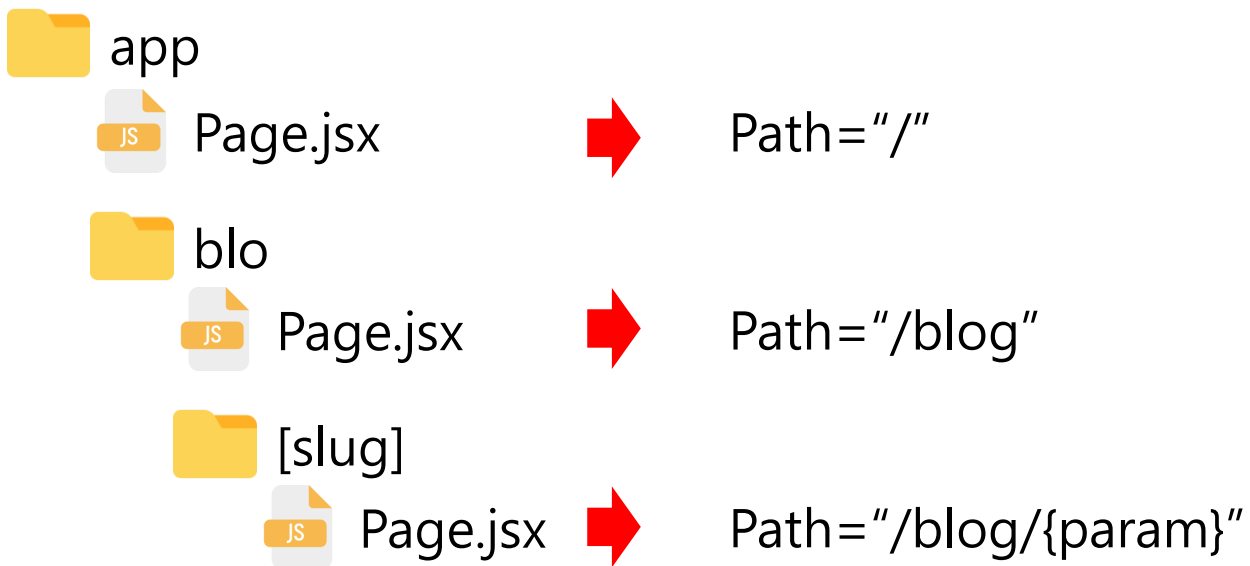
- src 폴더에는 실제 코드들이 존재
- app 폴더에는 하나의 페이지를 구성하는 파일들이 존재함
- page.js와 layout.js를 중점적으로 확인해 보자



Routing

Routing

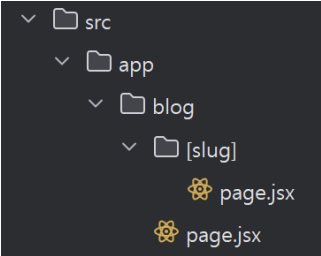
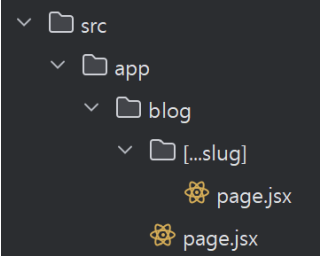
- Next.js에서는 폴더로서 경로 지정이 가능
- React.js보다 쉽게 Routing 이 가능
- Props의 params와 searchParams를 통해 parameter를 쉽게 받을 수도 있음



Routing

☑ Next.js 의 Router 기능을 활용해 보자

- 폴더명에 따른 parameter 수신 구조의 변화도 확인해 보자

폴더 구조	호출 url	Parameter 구조
app/blog/[slug]/page.jsx 	/blog/1	{slug:'1'}
app/blog/[...slug]/page.jsx 	/blog/2	{{slug:['2']}}
	/blog/food/2	{slug:['food','2']}
app/blog/[category]/[item]/page.jsx	/blog/food/2	{category:'food',item:'2'}

Routing

☑ Next.js 의 Router 기능을 활용해 보자

```
export default async function page(props){
  const param = await props.params;
  // [slug] 로 받는 경우...
  console.log(param);

  let list = <li key={param.slug}>{param.slug}</li>;

  // 폴더명이 [...slug] 로 변화하면 이후 모든 경로를 배열로 받는다.
  // 그렇게 되면 받는 형식이 달라져야 한다.
  if(Array.isArray(param.slug)){
    list = param.slug.map((item, idx) => {
      console.log(item);
      return <li key={idx}>{item}</li>
    });
  }
  return ( ... );
}
```



blog/11	slug:[11]
blog/food/11	slug:[food,11]
blog/food/2025/04/15	slug:[food,2025,04,15]

실습파일 : 14_router > src > app > blog > page.jsx

Image 최적화

Image 최적화

- Next.js에서는 img보다는 Image 태그를 사용하는 것을 권장함
- 노출된 이미지를 저장 후 차이를 확인해 보자

- 원본크기로 저장
- 확장자 : 원본 파일 형태

Image 태그의 특징

자동적으로 이미지를 기기에 맞는 사이즈로, 최신 형식인 WebP와 AVIF로 전달

placeholder = 'blur' 설정 후 blurDataURL을 지정하면, 빌드시 생성된 작은 사이즈의 이미지가 해당 영역을 차지하고 있다가 이미지 로딩이 끝나면 대체

외부 URL 이미지 사용시,
사이즈 속성 width, height 필수임.

- 화면에 최적화된 사이즈로 저장
- 확장자 : Webp

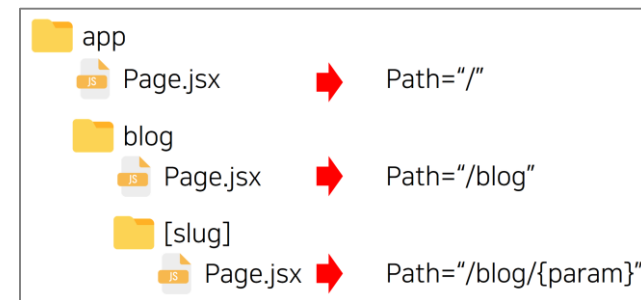
```
export default function Page(props) {  
  return (  
    <div>  
        
      <hr/>  
      <Image src="/incheon.jpg"  
        placeholder="blur"  
        blurDataURL="/incheon_small.jpg"  
        width={1027} height={768}  
        alt="incheon.jpg" />  
    </div>  
  );  
}
```

실습파일 : 15_image > src > app > page.js

Summary

☑ Routing

- Next.js 에서는 폴더로서 경로 지정이 가능
- Props의 params와 searchParams를 통해 parameter를 쉽게 받을 수도 있음



☑ Image

- Image 태그 사용시 화면에 최적화된 사이즈로 저장, 최신 형식인 WebP와 AVIF로 전달
- blurDataURL을 지정하면, 로딩 시 저해상도 이미지로 대체
- 외부 URL이미지 사용시, 사이즈 속성 width, height 필수

05.

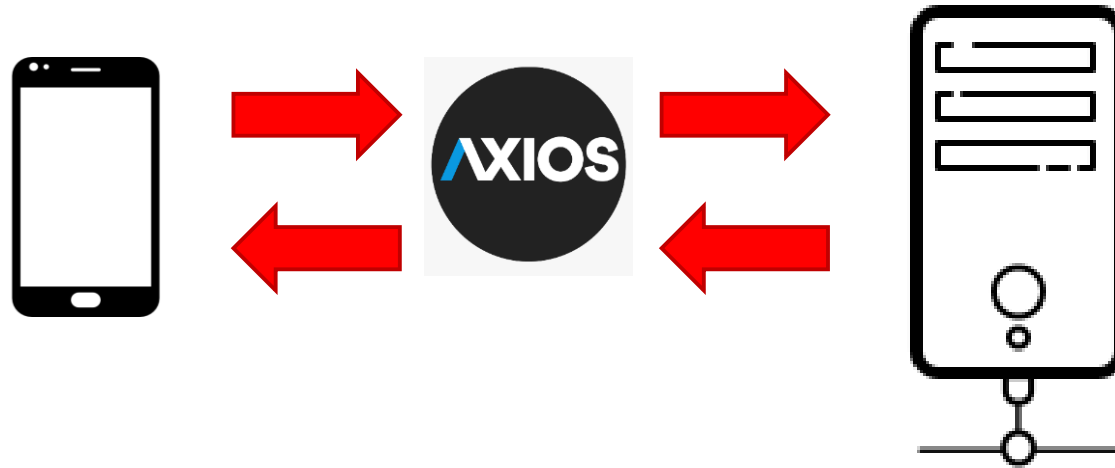
Network 기능 활용과 로그인 인증 처리

Network

✓ Network

- React.js는 서비스 제작 시 서버와의 통신이 필요한 경우가 많음
- axios 라이브러리 사용 시 보다 편리하게 서버와의 통신이 가능

```
npm install axios //npm 명령어를 활용한 axios 설치
```



Network

☑ 서버 통신 library를 활용하여 외부 서버와 통신해 보자

- ① axios를 통해 받아온 데이터를 list state에 저장
- ② 저장된 list state 를 Post component에 전달
- ③ props 로 전달된 list 로 UI 를 생성
- ④ 조건에 따라 각기 다른 내용을 rendering

```
export default function SendList({url}) {  
  const [list, setList] = useState([]);  
  ① const send= async ()=>{  
    let {data} = await axios.get(url);  
    setList(data);  
  }  
  
  return(  
    <div>  
      ② <button onClick={send}>전송</button>  
      <Post list={list}/>  
    </div>  
  );  
}
```

```
function Post({list}) {  
  let post_list = list.map(post=>(  
    <li key={post.id}>{post.title}</li>  
  ));  
  
  return(  
    <ul>  
      ④ {post_list.length>0 ?  
        post_list : '불러온 POST가 없습니다.'}  
    </ul>  
  );  
}
```

실습파일 : 16_axios > src > app > SendList.jsx

Network 실습(준비)

☑ 1단계 : 서버와의 통신을 위한 API 문서를 확인해 보자

항목	설명	예시
url	호출할 서버의 기본 주소	/detail/{idx}
method	요청 시 사용할 method(GET,POST 등)	GET
예시	요청 예시	/detail/123
설명	해당 기능에 대한 설명	특정 idx의 게시글 가져오기
응답	요청 시 수신하게 될 응답 값 예시	{idx:123, subject:'간단한제목',content:'간단한 내용',...}

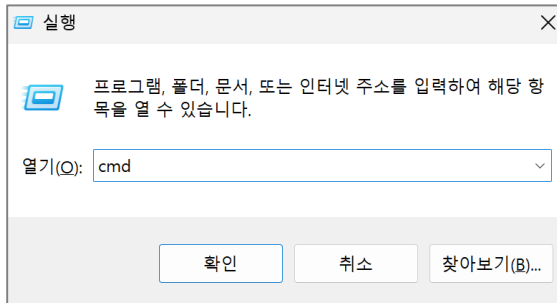
실습파일 : 17_board > API_DOC.txt

Network 실습(준비)

☑ 2단계 : 주어진 문서대로 서버가 잘 동작하는지 확인해 보자

- Spring Boot로 제작된 서버를 Terminal에서 구동해 보자

1.  + R 로 실행 창을 열고 cmd 명령어 실행



2. server.war가 있는 곳으로 이동 후 war 파일 실행

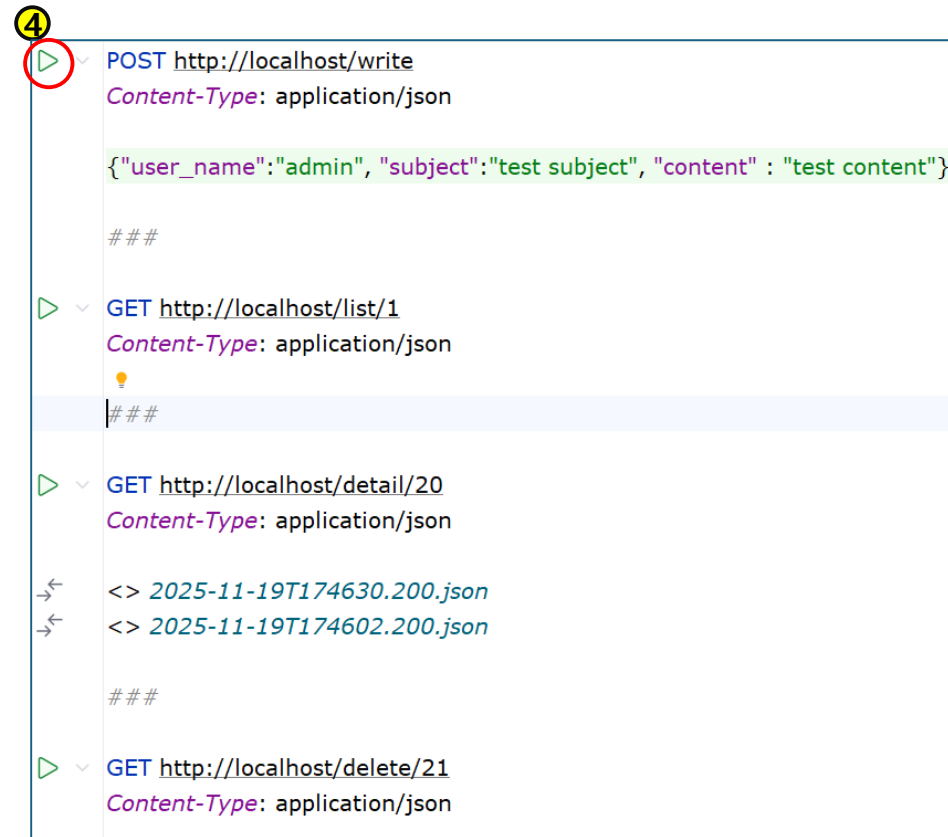
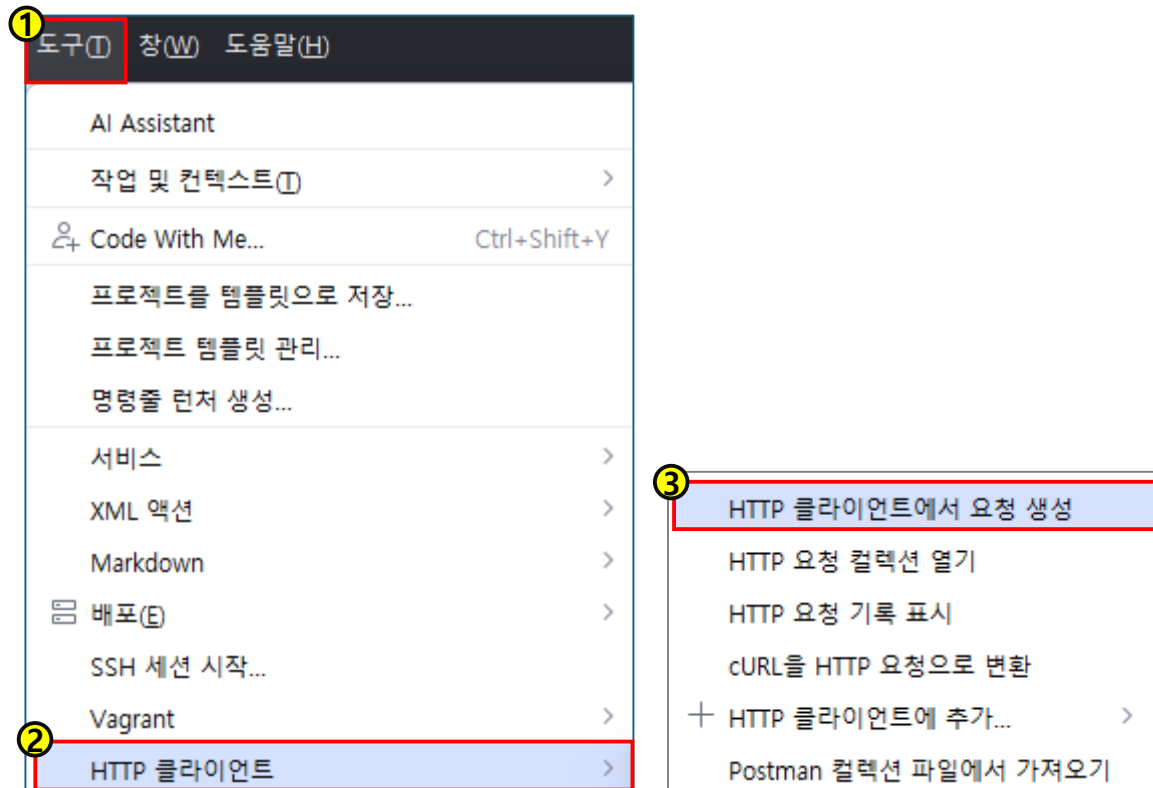
```
C:\Users\waivle> cd ₩  
C:\₩> cd 17_board₩server.war 위치 경로  
C:\₩> java -jar server.war
```

실습파일 : 17_board > server.war

Network 실습(준비)

2단계 : 주어진 문서대로 서버가 잘 동작하는지 확인해 보자

- Web Storm 에서 "도구 > HTTP 클라이언트 > HTTP 클라이언트 에서 요청 생성"
- 문서대로 잘 동작하는지 결과를 확인해 보자



실습파일 : 17_board > TEST_API.txt

Network 실습문제 (심화)

☑ 게시판 서비스의 Client App을 작성해 보자

- 다음 결과 화면과 스텔레톤 코드의 주석을 보고 리스트 페이지를 구현해 보자

리스트 가져오기

글쓰기

5 : 공지드립니다.[0]

관리자

3 : 테스트2[0]

유빛나

2 : 테스트[0]

유빛나

1 : test subject[0]

admin

```
export default function PostList({page}) {  
  const [posts, setPosts] = useState([]);  
  // 1. 시작하자마자 리스트를 호출하고  
  useEffect(()=>{  
    // 2. makeItem() 호출  
  }, []);  
  
  const makeItem=(data) => {  
    // 3. 받은 데이터로 list 를 만들고... state 에 저장  
    setPosts(content);  
  };  
  
  return(  
    <div>  
      {posts} {/*state 인 posts 출력*/}  
    </div>);  
}
```

실습파일 : 17_board > 17_board > src > app > PostList.jsx

Network 실습문제 (심화)

☑ 게시판 서비스의 Client App을 작성해 보자

- 다음 결과 화면과 스켈레톤 코드의 주석을 보고 상세보기 페이지를 구현해보자

상세보기

작성자 : 관리자

조회수 : 0

제목 : 공지드립니다.

간단한 공지사항 전달 드려요

[리스트](#)

삭제

```
export default function Post({idx}){
  const [post,setPost] = useState(null);
  // 1. 시작하자마자 리스트를 호출하고
  useEffect(()=>{
    // 2.makeHTML() 호출
  },[]);

  const makeHTML = (info) => {
    // 3. 받은 info 로 페이지를 만들어서 state 에 저장
  }

  return(
    <div>{post}</div> {/*state 인 post 출력*/}
  );
}
```

실습파일 : 17_board > 17_board > src > app > detail > slug > Post.jsx

Network 실습문제 (심화)

✓ 게시판 서비스의 Client App을 작성해 보자

- 다음 결과 화면과 스켈레톤 코드의 주석을 보고 글쓰기 페이지를 구현해보자

리스트

글쓰기

```
export default function Write() {  
  
  const [info, setInfo] = useState({user_name:"", subject:"",content:"});  
  const input=(key,e)=>{  
    // 1. 입력 내용 state 에 저장 로직  
  }  
  const write = async()=>{  
    // 2. 글쓰기 저장 요청 로직  
  }  
  return (  
    <div className="write">{/* 글쓰기 UI 출력*} </div>  
  );  
}
```

실습파일 : 17_board > 17_board > src > app > write > page.jsx

로그인 인증



JWT

- 로그인 인증 등 통신 시 유지되어야 하는 정보를 보안을 유지하면서 저장할 수 있는 무언가가 필요
- 여러 방법 중 가장 널리 활용되는 JWT(JSON Web Token)은 client와 server 사이 통신 시 사용하는 토큰
- 주로 권한 정보 확인 시 자주 사용
- 암호화된 문자열이기에 쉽게 읽을 수 없음
- 구성은 header, payload, signature로 나뉘며 이들 사이는 dot(.) 으로 구분



구분	내용
header	어떤 알고리즘을 사용하고, 어떤 토큰을 사용할 것인지에 대한 선언
payload	전달하려는 정보
signature	header + payload 와 함께 비밀키로 서명한 내용이며 이걸로 변조 여부를 검사한다.

로그인 인증

☑ JWT 를 활용하여 로그인 처리 및 검증을 진행해 보자

- 먼저 next.js 앱과 통신할 서버를 구동시키자. (Network 실습(준비) 2단계 79p~ 참조)

```
java -jar server.war
```

서버는 18_jwt/server.war 를 사용할 것.

• 로그인 처리

```
const login=async()=>{  
  ① let {data} = await axios.post("http://localhost/login",info);  
  
  if(data.success){  
    ② sessionStorage.setItem("token",data.token);  
    sessionStorage.setItem("id",info.id);  
    location.href="/list/1";  
  }else{  
    alert('아이디 또는 비밀번호를 확인해 주세요!');  
  }  
}
```

로그인 요청 후 성공 시 token 수신

sessionStorage 에 수신한 token 저장

실습파일 : 18_jwt/18_jwt/src/app/page.js

로그인 인증

- 로그인 검증 요청

```
const getDetail=async(idx)=>{  
  const id = sessionStorage.getItem("id");  
  ① const token = sessionStorage.getItem("token");  
  ② const {data} = await axios.get(url,  
    {headers:{Authorization:token}});  
  setInfo(data.detail);  
  if(data.photos.length>0){  
    setList(<PhotoList photos={data.photos}/>);  
  }  
}
```

sessionStorage에서 token 획득

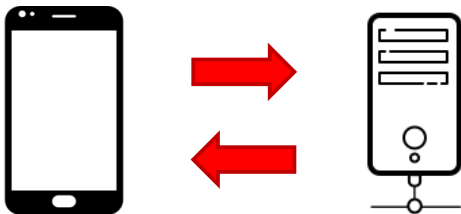
Header에 추가 후 서버로 전송

실습파일 : 18_jwt > 18_jwt > src > app > detail > [slug] > page.js

Summary

☑ Network

- axios 라이브러리를 활용하면 서버 통신 가능
- 서버 통신 APP을 만들기 위해 API 문서 확인 필요



☑ 로그인 인증

- Session은 서버내부에서 공유되는 저장 장소
- 서로 다른 서버에서는 session을 사용 할 수 없음
- 그래서 보안을 유지하면서 주요 정보를 저장할 JWT 를 사용

header . payload . signature

06.

라이브러리 활용과 데이터 공유, 빌드

외부 라이브러리

☑ 외부 라이브러리 추천 및 소개

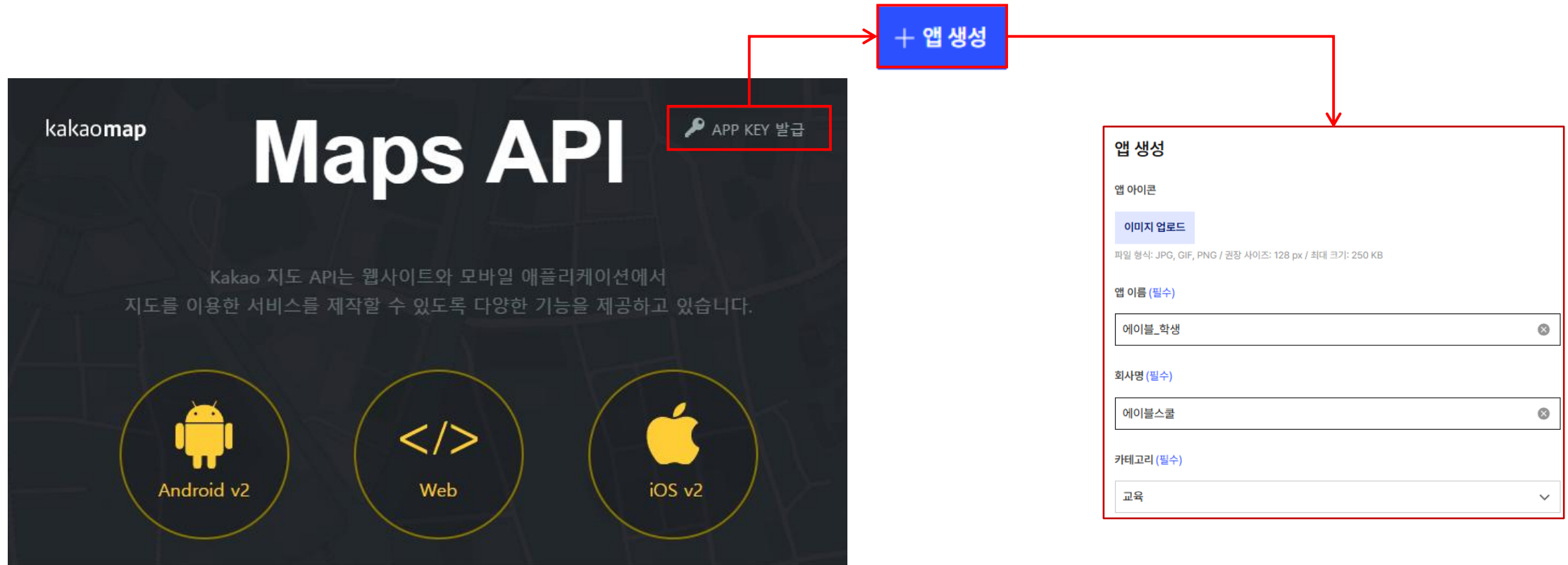
- 프로젝트 진행 시 자주 사용되는 몇몇 기능들에 대해서는 외부 Library를 활용해야 함

라이브러리	설명
kakao map	map Library
bootstrap	UI Design Library
chart.js	Chart Library

외부 라이브러리

☑ 카카오 맵 - 1단계 : 키 발급

- <https://apis.map.kakao.com/> 에서 회원가입 후 key 발급 진행
- 이후 앱 생성 진행



외부 라이브러리

☑ 카카오 맵 - 1단계 : 키 발급

- 앱 생성 후 비즈니스 앱 등록을 선택하면 앱 키 확인 가능
- 그리고 Web 플랫폼에서 사용할 도메인을 등록
- Map을 활용하기 위한 JavaScript 키를 미리 획득해 놓자



앱 키 문서 확인

앱 키는 API 호출 시 앱 인증을 위해 사용됩니다. 플랫폼에 따라 사용해야 하는 앱 키가 다릅니다.

네이티브 앱 키		복사	재발급
REST API 키		복사	재발급
JavaScript 키		복사	재발급
어드민 키		복사	재발급

외부 라이브러리

☑ 카카오 맵 - 2단계 : 맵 적용

- 마지막으로 카카오맵 상태를 ON 으로 변경

The image shows the Kakao Maps settings interface. On the left, a vertical menu lists various settings: '제품 설정' (Product Settings), '카카오 로그인' (Kakao Login), '비즈니스 인증' (Business Authentication), '카카오톡 메시지' (KakaoTalk Message), '카카오맵' (Kakao Maps), and '푸시 알림' (Push Notification). The '카카오맵' option is highlighted with a red box. A red arrow points from this box to the right panel. The right panel has a title '카카오맵' with a '문서 확인' (Check Document) link. Below the title, it says '카카오맵의 주요 설정을 확인하고 관리할 수 있습니다.' (You can check and manage the main settings of Kakao Maps). Under the '사용 설정' (Usage Settings) section, which also has a '문서 확인' link, it says '카카오맵의 사용 여부를 선택할 수 있습니다.' (You can select whether to use Kakao Maps). At the bottom of this section, there is a '상태' (Status) label followed by a toggle switch that is currently turned ON.

카카오맵 [문서 확인](#)

카카오맵의 주요 설정을 확인하고 관리할 수 있습니다.

사용 설정 [문서 확인](#)

카카오맵의 사용 여부를 선택할 수 있습니다.

상태 ☒ ON

외부 라이브러리

☑ 2단계 : 맵 적용

- 화면에 맵을 적용해 보자

```
export default function Layout({ children }) {  
  
  const api_key = '발급받은 키';  
  
  return (  
    <html lang="ko">  
      <head>  
        <meta charset="utf-8" />  
        <script src={`url?appkey=${api_key}&autoload=false`}></script>  
      </head>  
      <body>  
        {children}  
      </body>  
    </html>  
  );  
}
```

이전 단계에서 발급받은 키값 입력

키를 포함한 Kakao map을 호출하기 위한 URL 생성

실습파일 : 19_map > src > app > layout.js

외부 라이브러리

☑ 2단계 : 맵 적용

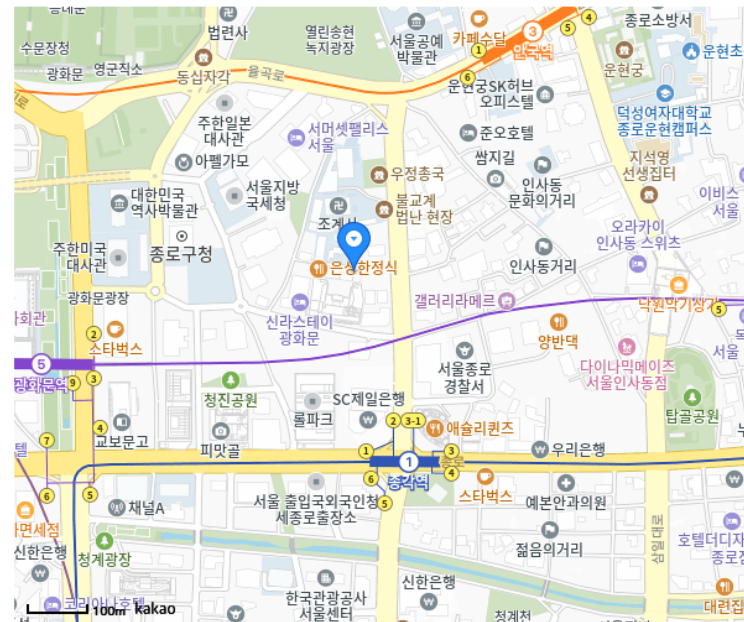
- 화면에 맵을 적용해 보자

```
export default function HomePage(){
  const [msg, setMsg] = useState('');
  const container = useRef(null);
  useEffect(() => {
    // 카카오맵 설정
    // 최초 마커 등록
    // 이벤트 등록
  }, []);

  return (
    <>
      <div id="map" style={{width:"100%",height:"500px"}}
        ref={container}></div>
      <p id="msg">{msg}</p>
    </>
  );
}
```

실습파일 : 19_map > src > app > page.js

• 결과 화면



클릭한 위치의 위도는 37.5730236332742, 경도는 126.98215968778834 입니다.

외부 라이브러리

```
useEffect(() => {
  const { kakao } = window; // window 전역 객체에서 kakao 가져오기
  if (!kakao || !kakao.maps) return; //카카오 스크립트가 아직 로드되지 않았다면 실행 중지
  kakao.maps.load(()=>{
    // 카카오 맵 설정
    const mapOption = {
      center: new kakao.maps.LatLng(37.57190029146425, 126.98715765847491),
      level: 3
    };
    let map = new kakao.maps.Map(container.current, mapOption);
    // 최초 마커 등록
    let marker = new kakao.maps.Marker({
      position: map.getCenter(),
    });
    marker.setMap(map);
    // 이벤트 등록
    kakao.maps.event.addListener(map, 'click', function (event) {
      console.log('evt',event);
      let latLan = event.latLng;
      marker.setPosition(latLan); // 특정 위도, 경도로 마커 이동
      let msg = '클릭한 위치의 위도는 '+latLan.getLat()+', 경도는 '+latLan.getLng()+ ' 입니다.';
      setMsg(msg);
    });
  });
}, []);
```

(참고) 올바른 api_key가 지정되지 않은 상태로 호출하거나, 스크립트 로드 지연 발생 시 오류가 발생할 수 있음.

실습파일 : 19_map > src > app > page.js

외부 라이브러리

Bootstrap

- Bootstrap은 손쉽게 퀄리티 있는 디자인을 만들어 낼 수 있는 library
- 반응형 웹을 지원하고 있어 개발자가 간단한 서비스를 만들 때 유용함
- 다양한 사용법은 아래를 참고하자

```
//npm 명령어를 활용한 react-bootstrap & bootstrap 설치  
npm install react-bootstrap bootstrap
```

사용법 참조 : <https://react-bootstrap.github.io/docs/components/alerts>

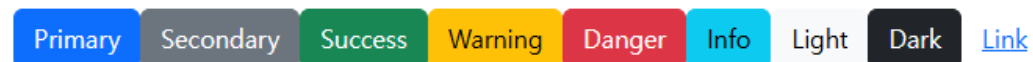
외부 라이브러리

☑ Bootstrap 을 활용한 UI 생성 법에 대해서 알아보자

```
export default function Home(){
  return(
    <div>
      <ButtonToolbar>
        <Button variant="primary">Primary</Button>
        <Button variant="secondary">Secondary</Button>
        <Button variant="success">Success</Button>
        <Button variant="warning">Warning</Button>
        <Button variant="danger">Danger</Button>
        <Button variant="info">Info</Button>
        <Button variant="light">Light</Button>
        <Button variant="dark">Dark</Button>
        <Button variant="link">Link</Button>
      </ButtonToolbar>
    </div>
  );
}
```

실습파일 : 20_bootstrap > src > app > page.jsx

- 결과 화면



외부 라이브러리

MUI X Charts 활용

- 외부 라이브러리를 활용하여 차트를 그려보자!
- 디자인과 API의 친절도 등을 고려하여 선택하자.

```
//npm 명령어를 활용한 @mui/material @emotion/react @emotion/styled 설치  
npm install @mui/material @emotion/react @emotion/styled
```

```
//npm 명령어를 활용한 @mui/x-chart 설치  
npm install @mui/x-charts
```

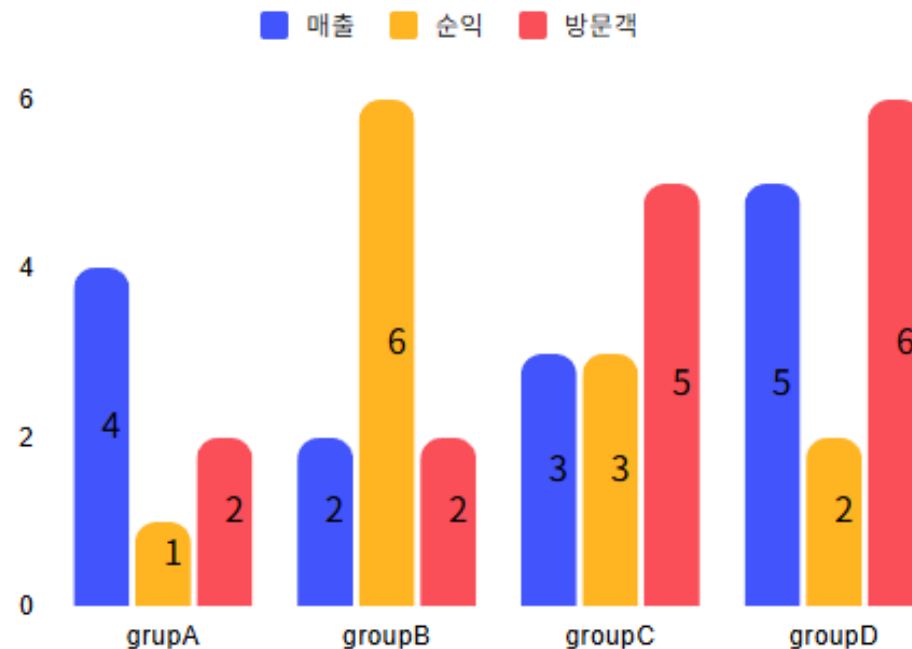
참조 사용법 : <https://mui.com/x/react-charts/getting-started/>

외부 라이브러리

☑ MUI X Charts를 활용하여 차트를 생성해 보자 – bar chart

```
<BarChart
  series={[
    {data:[4,2,3,5], label:'매출'},
    {data:[1,6,3,2], label:'순익'},
    {data:[2,2,5,6], label:'방문객'}
  ]} /* 차트에 사용할 데이터*/
  xAxis={[{data:['groupA','groupB','groupC','groupD'],
    scaleType:'band'}}] /*X 축에 묶을 그룹 */
  width={500} /*표의 가로 크기*/
  height={300} /*표의 세로 크기*/
  barLabel={"value"} /*Bar 에 표시될 내용*/
  borderRadius={10} /*Bar 모서리의 둥글기*/
  grid={{horizontal:true}} /*눈금 표시 여부*/
/>
```

실습파일 : 21_chart > src > bar > page.jsx

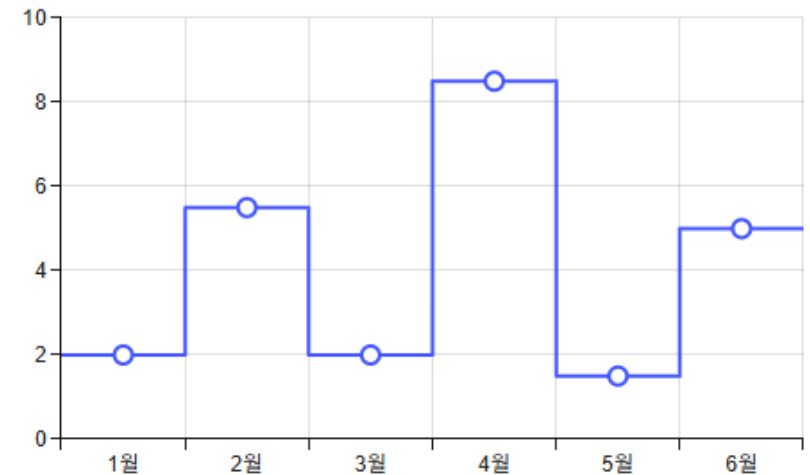


외부 라이브러리

☑ MUI X Charts 를 활용하여 차트를 생성해 보자 – line chart

```
<LineChart
  /*scaleType:'band' 가 있어야 data 에 문자열이 들어갈 수 있음*/
  xAxis={[{scaleType:'band'
    ,data:['1월', '2월', '3월', '4월', '5월', '6월']}]}}
  series={[
    {
      data: [2, 5.5, 2, 8.5, 1.5, 5],
      curve: 'step' /*차트에 사용할 데이터 및 선 스타일*/
    },
  ]}
  width={500} /*차트 가로 크기*/
  height={300} /*차트 세로 크기*/
  grid={{ vertical: true, horizontal: true }}
/>
```

실습파일 : 21_chart > src > line > page.jsx

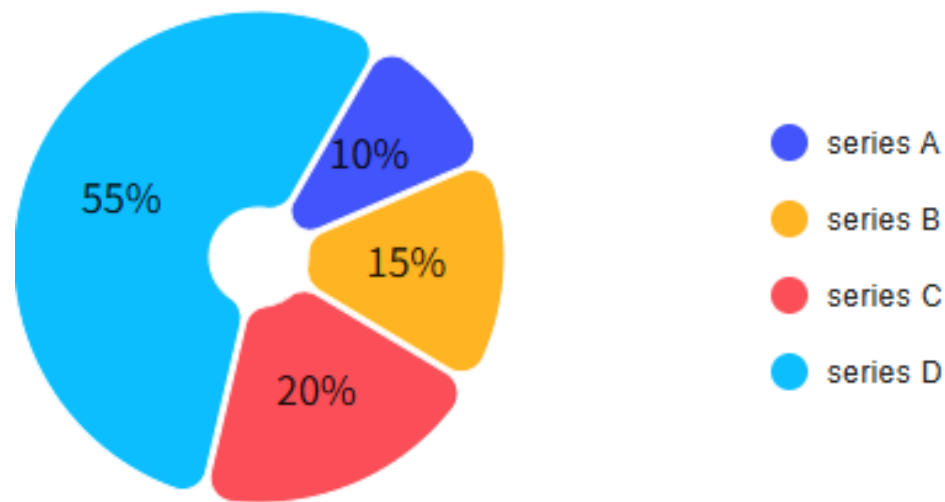


'catmullRom', 'linear', 'monotoneX',
'monotoneY', 'natural', 'step',
'stepBefore', 'stepAfter'

외부 라이브러리

☑ MUI X Charts 를 활용하여 차트를 생성해 보자 – pie chart

```
<PieChart
  series={[ /*차트에 사용할 데이터*/
    {data: [
      {value:10,label:'series A'},
      {value:15,label:'series B'},
      {value:20,label:'series C'},
      {value:55,label:'series D'},
    ]},
    /*파이차트 스타일 지정*/
    innerRadius:20, outerRadius:100,
    paddingAngle:1, cornerRadius:10,
    startAngle:30, endAngle: 390,
    /*파이차트 라벨 표시 형식*/
    arcLabel:item =>`${item.value}%`
  ]}
  /*차트 가로세로 크기 지정*/
  width={400} height={200}
/>
```



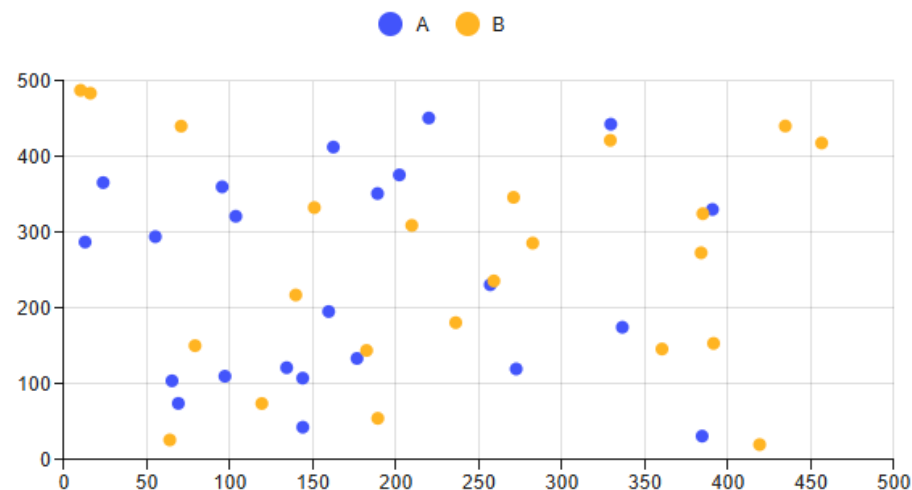
실습파일 : 21_chart > src > pie > page.jsx

외부 라이브러리

☑ MUI X Charts 를 활용하여 차트를 생성해 보자 – scatter chart

```
<ScatterChart
  series={[
    {label: 'A', data:data.map(d=>({x:d.x1, y:d.y1}))),},
    {label: 'B', data:data.map(d=>({x:d.x2, y:d.y2}))),},
  ]} /* 차트에 사용할 데이터 */
  width={600} /* 차트 가로 크기*/
  height={300} /* 차트 세로 크기*/
  /*눈금 표시 여부*/
  grid={{ vertical: true, horizontal: true }}
/>
```

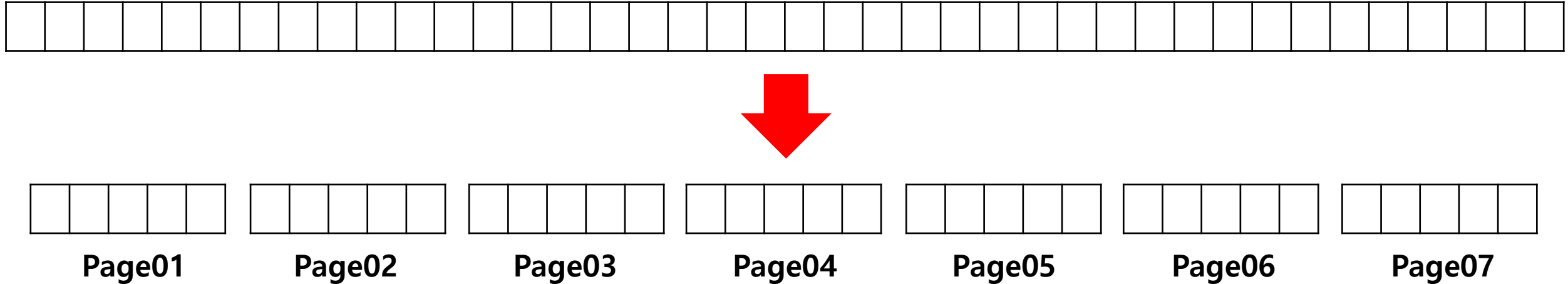
실습파일 : 21_chart > src > scatter > page.jsx



외부 라이브러리

☑ pagination

- 많은 데이터를 한 번에 불러오는 것은 DB에 과부하를 줄 수 있음
- 그렇기 때문에 paging을 통해 n개씩 나눠서 가져오는 paging 처리를 하고 있음
- Paging 처리 UI 를 생성해주는 라이브러리를 활용해 보자



```
//npm 명령어를 활용한 @mui/material @emotion/react @emotion/styled 설치  
npm install @mui/material @emotion/react @emotion/styled
```

외부 라이브러리

☑ paging 처리를 적용해 보자

- client와 통신할 서버를 구동해주자 ([Network 실습 \(준비\)](#) 2단계 79p~ 참조)

`java -jar server.war` 서버는 18_jwt/server.war 를 사용할 것.

```
import Pagination from "@mui/material/Pagination";
import Stack from "@mui/material/Stack";

<Stack spacing={2}>
  <Pagination count={pages} //전체 페이지 수
    color={"primary"} //테마 색상
    variant={"outlined"} //페이지 넘버 외관선 종류
    shape={"rounded"} //페이지 넘버 모양
    siblingCount={0} //중간 넘버 양쪽에 표시할 개수
    onChange={changePage} //Click 시 변경 시 실행 함수 이름
  />
</Stack>
```

실습파일 : 22_pagination/src/app/list/[slug]/page.jsx

로그인

ID	아이디를 입력 하세요	로그인
PW	비밀번호를 입력 하세요	

- 로그인하여 페이지에 진입
- ID : admin
- PW : pass

결과 화면

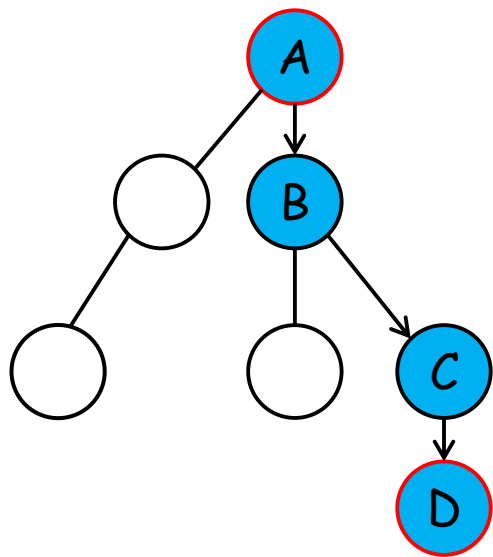
<	1	2	3	...	10	>
---	---	---	---	-----	----	---

Reference : <https://mui.com/material-ui/react-pagination/>

데이터 공유 방식

☑ React.js 의 데이터 공유 방식

- React.js에서는 데이터가 부모 컴포넌트에서 자식 컴포넌트로 props를 통해 단방향으로 전달
- D가 A의 데이터를 사용하기 위해서는, B와 C를 차례로 거쳐 데이터가 전달되어야 함
- 이때 계속해서 아래로 내려가야 하므로 **Prop Drilling**이라고 부름



} 불필요한 중간 컴포넌트들이
데이터를 전달만 하는 과정에
서
유지보수가 어려워지는 문제

데이터 공유 방식

☑ App에서 Island, 그리고 Third까지 데이터를 보내는 과정을 살펴보자

```
export default function App(){  
  return (  
    <div>  
      <First item="App Data"/>  
      <Island item="App Data"/>  
    </div>  
  );  
}
```

실습파일 : 23_data_flow > src > app > page.jsx

<Second item={item} />

<Third item={item} />

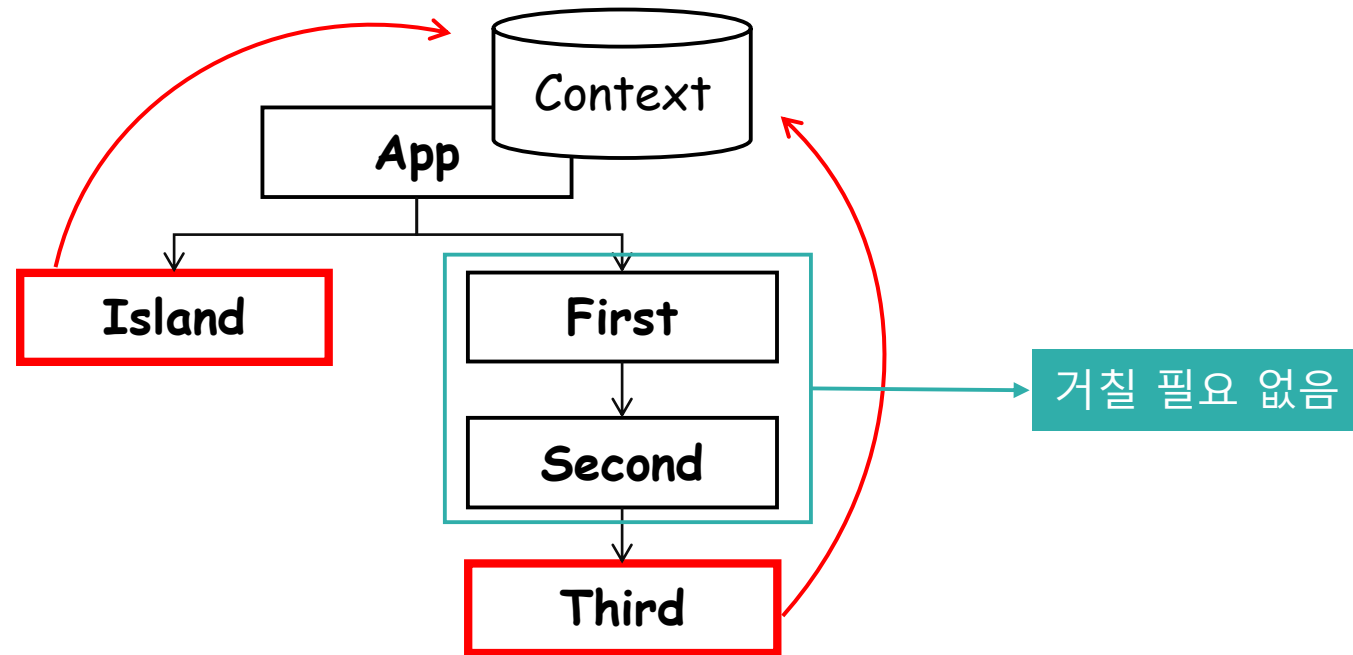
- 결과 콘솔

First Component / 거쳐가는 곳1 : App Data
Second Component / 거쳐가는 곳2 : App Data
Third Component / 최종 도착지 : App Data
Island Component / 최종 도착지 : App Data

데이터 공유 방식

☑ Context 객체

- 이 경우 모두가 사용할 수 있는 저장소가 있다면?
- Context는 생성한 데이터를 모든 자식 컴포넌트에서 받아볼 수 있는 저장소 개념



데이터 공유 방식

☑ Context 객체

- Context를 활용하여 Island와 Third 컴포넌트에만 데이터를 전달해 보자

```
export default function App(){  
  return (  
    <DataContext.Provider value="Context App  
Data">  
      { /*item 을 Third 에 전달하는 과정*/}  
      <First/>  
      <Island/>  
    </DataContext.Provider>  
  );  
}
```

```
function Third() {  
  const data = useContext(DataContext);  
  return (  
    <div>  
      Third Component / 최종 도착지 : {data}  
    </div>  
  );  
}
```

• 결과 콘솔

First Component

Second Component

Third Component / 최종 도착지 : Context App Data

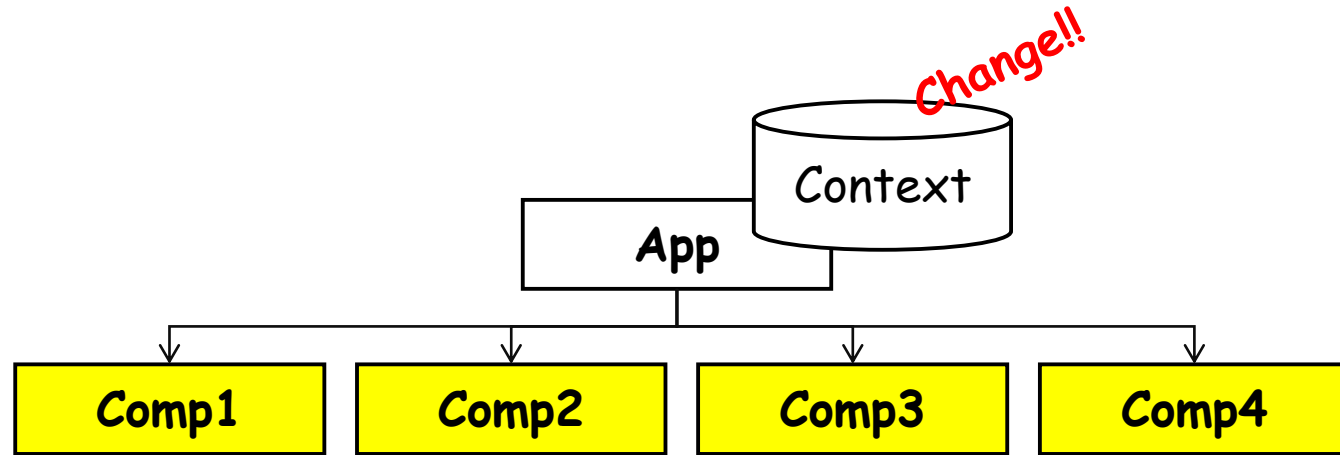
Island Component / 최종 도착지 : Context App Data

실습파일 : 24_context/src/app/page.jsx

데이터 공유 방식

☑ Context 객체의 단점

- Context API에도 단점은 존재함
- Context를 공유하는 Component 들은 Context 값 변경 시 모두 렌더링 됨



데이터 공유 방식

☑ Context API 사용 시 단점을 알기 위해 아래 구조의 컴포넌트를 만들어 보자

```
'use client'
import {useContext, useMemo} from "react";
import {UserContext} from "../page";

export default function UserComponent(props){
  const {info, setInfo} = useContext(UserContext); // context 객체에 저장된 값을 불러온다.
  const inputStr=(field, e) => { // onchange 이벤트 발생시 전달되는 값들
    // context에 저장된 info 값을 복사해둔 채로 받아온 키와 값을 추가한다.
    setInfo({...info, [field]: e.target.value});
  }
  console.log('User Component Rendering ');
  return (
    <>
      /* onchange 이벤트 발생시 inputStr 실행*/
      <p>ID : <input type="text" value={info.id} onChange={(e)=>{inputStr('id',e)}}/></p>
      <p>PW : <input type="text" value={info.pw} onChange={(e)=>{inputStr('pw',e)}}/></p>
    </>);
}
```

실습파일 : 25_render > src > app > UserComp.jsx

데이터 공유 방식

☑ Context API 사용 시 단점을 알기 위해 아래 구조의 컴포넌트를 만들어 보자

```
'use client'
import {PostContext} from "../page"
import {useContext} from "react";

export default function PostComponent(props) {
  const {cnt, setCnt} = useContext(PostContext); // context 객체에 저장된 값을 불러온다.
  console.log('PostComponent Rendering');
  return (
    <>
      {/* context 객체에서 불러온 값 사용 */}
      <h3>Post 에 대한 조회수 : {cnt}</h3>
      <button onClick={()=>{setCnt(cnt+1)}}>좋아요!</button>
    </>
  );
}
```

실습파일 : 25_render/src/app/PostComp.jsx

데이터 공유 방식

☑ Context API 로 인한 불필요한 rendering 상황

- UserComponent에서 입력을 통해 rendering을 일으키고 log를 확인해보자
- 사용하지 않는 context의 값이 수정되어도 두 컴포넌트 모두 렌더링 됨

• UserComp

```
export default function UserComponent(props){  
  const {info, setInfo} = useContext(UserContext);  
  ...  
  console.log('User Component Rendering');  
  return (...);  
}
```

• PostComp

```
export default function PostComponent(props) {  
  const {cnt, setCnt} = useContext(PostContext);  
  console.log('PostComponent Rendering');  
  return (...);  
}
```

UserComp	ID : admin PW :
PostComp	Post 에 대한 조회수 : 0 좋아요!

User Component Rendering [UserComp.jsx:14](#)
PostComponent Rendering [PostComp.jsx:9](#)

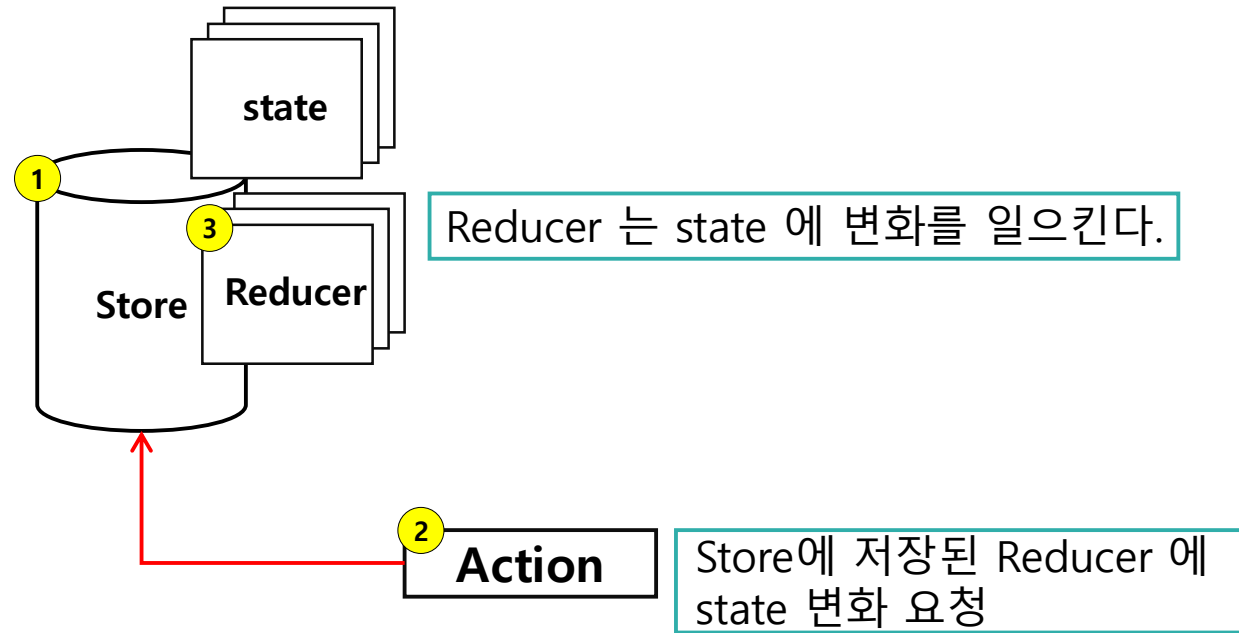
데이터 공유 방식

✓ Redux 개념

- 위에서 살펴본 prop drilling 과 Context re-rendering 이슈를 해결하기 위해 Redux 를 사용
- Redux 는 **state 와 state 를 다루는 함수들을 따로 모아**, 일정한 **절차에 따라 쓰도록 관리**해주는 라이브러리

• Redux 구성 요소

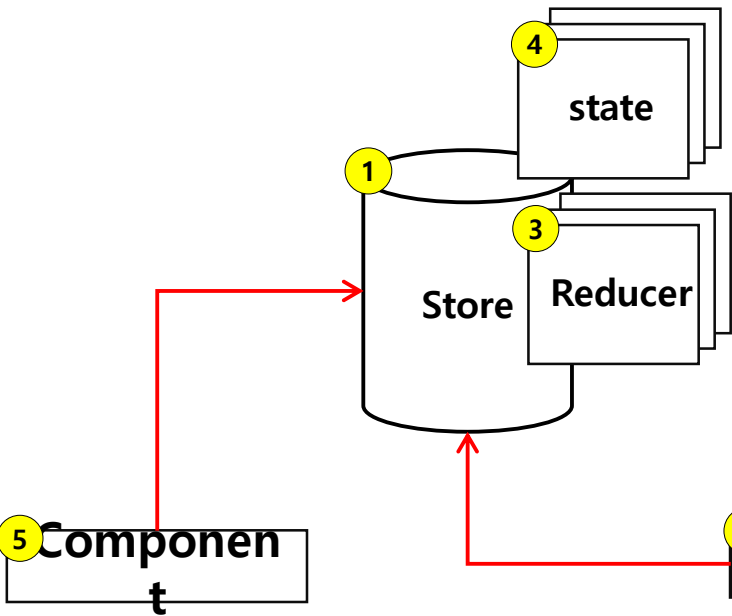
요소	설명
(1)Store	Application의 state를 저장하고 있는 객체
(2)Action	상태를 변화시키는 요청
(3)Reducer	State를 변화시키는 함수



데이터 공유 방식

✓ Redux 개념

- Redux 에서 store 에 저장된 상태(state)가 변화하는 흐름을 확인해 보자

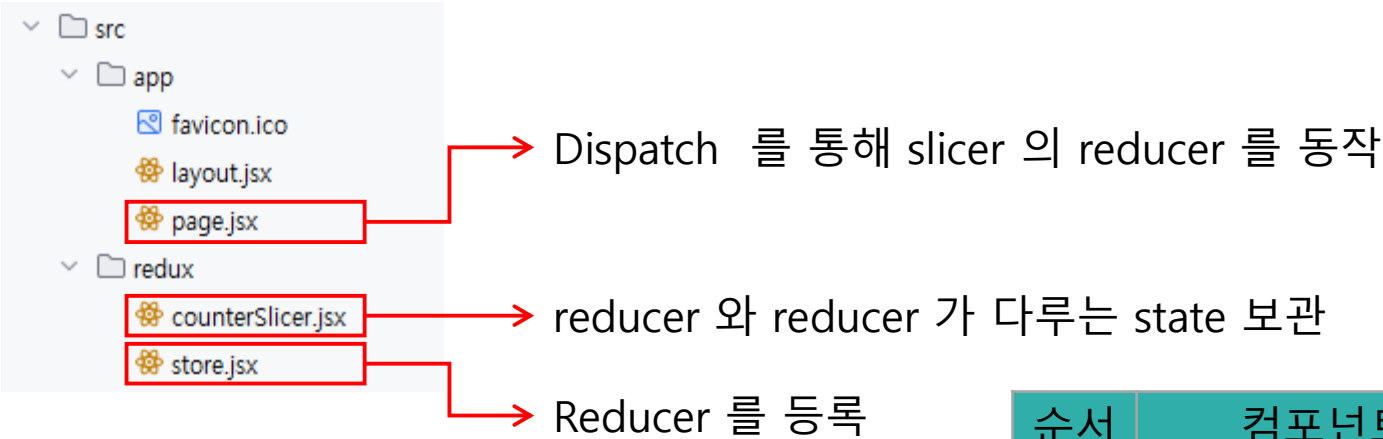


순서	설명
1	Store 에 state 와 reducer 를 등록한 상태
2	Store 에 Action 을 통해 state 변화 요청 - 이것을 action dispatch 라 부름
3	Store 에서 받은 Action을 Reducer 에 넘겨줌
4	요청에 따라 Reducer 에서 state 를 변경
5	Store 를 구독중인 Component 에서 state 변화에 따른 렌더링 실행

데이터 공유 방식

☑ Counter App 을 만들어 Redux의 주요 구성과 흐름에 대해 확인해 보자

```
npm install react-redux @reduxjs/toolkit //npm 명령어를 @reduxjs/toolkit 설치
```



순서	컴포넌트	설명
1	page.jsx	함수가 실행되면서 dispatcher로 특정 상태 변화 함수를 실행 요청
2	counterSlicer.jsx	등록된 reducer가 실행되고 state 값 수정
3	page.jsx	useSelector()를 활용 store에 변화를 감지 하고 해당 정보를 수신

데이터 공유 방식

1. Slicer 생성

```
const counterSlicer = createSlice({
  name: 'counter',
  initialState: { // 사용할 state
    value: 0
  },
  reducers: { // dispatch를 통해 수행될 reducer 들
    increment: (state, action) => {
      state.value = state.value + 1;
      return state;
    },
    decrement: (state, action) => {
      state.value = state.value - 1;
      return state;
    },
  },
});
export default counterSlicer.reducer;
```

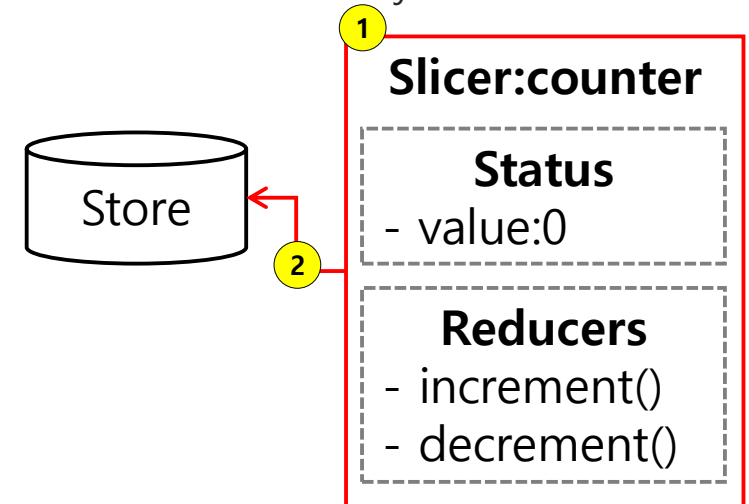
실습파일 : 26_counter > src > redux > counterSlicer.jsx

2. Store 에 reducer 등록

```
import counterSlicer from "../counterSlicer";

export const store = configureStore({
  reducer: {
    counter: counterSlicer
  }
});
```

실습파일 : 26_counter > src > redux > store.jsx



데이터 공유 방식

3. store에 저장된 state와 dispatch 활용

```
export default function Home(){
  const upHit = () => {
    store.dispatch({type:'counter/increment'}); // dispatch 를 호출해 state 값 변경
  };
  const downHit = () => {
    store.dispatch({type:'counter/decrement'});
  }
  let count = useSelector((state) => { // store 에 등록된 모든 slice의 state 정보 호출
    return state.counter.value;
  });
  return (
    <div>
      <h3>COUNT : {count}</h3>
      <button onClick={upHit}>증가</button>
      <button onClick={downHit}>감소</button>
    </div>
  );
}
```

실습파일 : 26_counter/src/app/page.jsx

3



Slicer:counter

Status

value:0

Reducers

increment()
decrement()

데이터 공유 방식 실습

☑ Redux를 활용하여 앞서 만든 board app에 적용해 보자

- client와 통신할 서버를 구동해 주자 ([Network 실습\(준비\)](#) 2단계 79p~ 참조)

`java -jar server.war` 서버는 18_jwt/server.war 를 사용할 것.

변경 기능	설명
login	로그인 처리 내용을 reducer 로 이동
list	리스트 호출 기능을 reducer 로 이동, 가져온 정보를 slice 의 state 에 저장
write	글쓰기 기능을 reducer 로 이동
detail	특정 글 호출 기능을 reducer 로 이동, 가져온 정보를 slice 의 state 에 저장
del	삭제 기능을 reducer 로 이동

데이터 공유 방식 실습문제 (심화)

- 다음 스켈레톤 코드의 주석을 보고 게시판과 관련된 slice를 구현해보자

```
const boardSlice = createSlice({
  name: 'board',
  initialState: {// state 값 초기화
    id: '', token: '', list: [], photos: [], pages: 0, detail: {}},
  },
  reducers: {
    list(state, action) { // 서버로 부터 받아온 데이터를 state 에 담아서 반환
      return state;
    },
    write(state, action) { ... }, // action 에서 받아온 데이터를 서버로 저장 요청
    del(state, action) { ... }, // action 에서 받아온 데이터를 서버로 삭제 요청
    detail(state, action) { // 서버로 부터 받아온 데이터를 state 에 담아서 반환
      ...
      return state;
    },
  },
});
```

실습파일 : 27_board > src > redux > boardSlice.jsx

데이터 공유 방식 실습문제 (심화)

- 다음 스켈레톤 코드의 주석을 보고 회원과 관련된 slice를 구현해보자

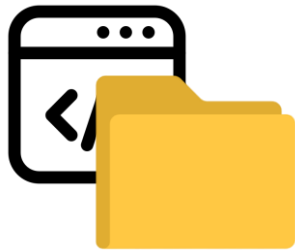
```
const memberSlice = createSlice({
  name: 'member',
  initialState: {},
  reducers: {
    login(state, action) {
      // 서버로 정보를 넘기고 받은 token 을 이용한 인증 처리
    },
  }
});
```

실습파일 : 27_board/src/redux/memberSlice.jsx

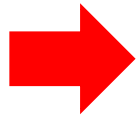
빌드



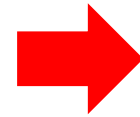
- Build란 개발한 소스가 실행 가능한 프로그램으로 변환하는 작업
- 그동안 개발환경(dev) 에서 실행되었기에 실제 빌드가 된 것이 아님
- 개발한 소스(source)를 빌드(build)를 통해 실행 가능한 프로그램 형태로 만드는 것을 배포(deploy) 라고 함



source



build



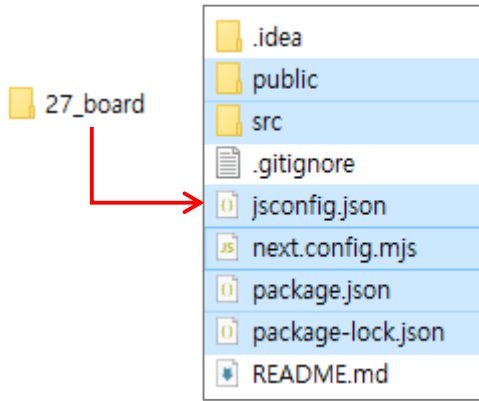
deploy

빌드

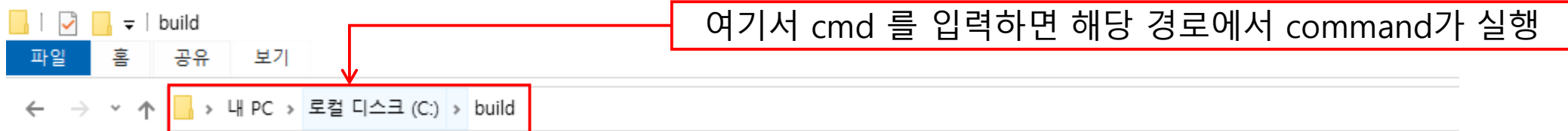
☑ 앞에서 만든 프로젝트를 build를 통해서 배포해 보자

1단계: 개발을 진행한 27_board폴더 에서 아래 내용을 복사 후 C:/build 폴더를 만들어 붙여넣기 실행

- 임의로 생성한 이름이므로 이름은 꼭 build 필요는 없다.
- 드라이브 파일 시스템이 FAT32 또는 exFAT 일 경우, 빌드 수행 안됨(주의!)



2단계: 이동한 곳에서 command 를 실행

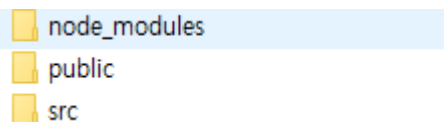


빌드

3단계: npm install 명령어로 package.json 의 의존성을 설치

```
C:\webuild>npm install
added 130 packages, and audited 131 packages in 11s
25 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
```

- 정상적으로 install 되면 node_modules 폴더 생성

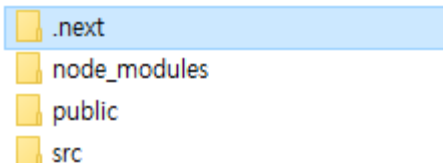


node_modules
public
src

4단계: npm run build 로 빌드 수행

```
C:\webuild>npm run build
> 27_board@0.1.0 build
> next build --turbo
  ▲ Next.js 15.5.4 (Turbo)
  Creating an optimized production build ...
  ✓ Finished writing to disk in 21ms
  ✓ Compiled successfully in 1950ms
  ✓ Linting and checking validity of types
  ✓ Collecting page data
  ✓ Generating static pages (6/6)
```

- 정상적으로 install 되면 .next 폴더 생성



.next
node_modules
public
src

빌드

5단계: npm run start 명령어로 서비스 실행

```
C:\#build>npm run start
```

```
> 27_board@0.1.0 start
```

```
> next start
```

```
▲ Next.js 15.5.4
```

```
- Local:      http://localhost:3000
```

```
- Network:    http://192.168.0.40:3000
```

```
✓ Starting...
```

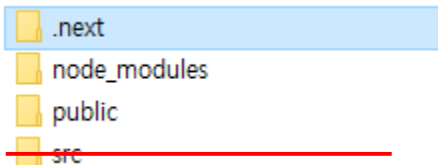
```
✓ Ready in 470ms
```

- 정상적 실행된 모습

로그인

ID	아이디를 입력 하세요	로그인
PW	비밀번호를 입력 하세요	

6단계: src의 소스 폴더 삭제(선택)



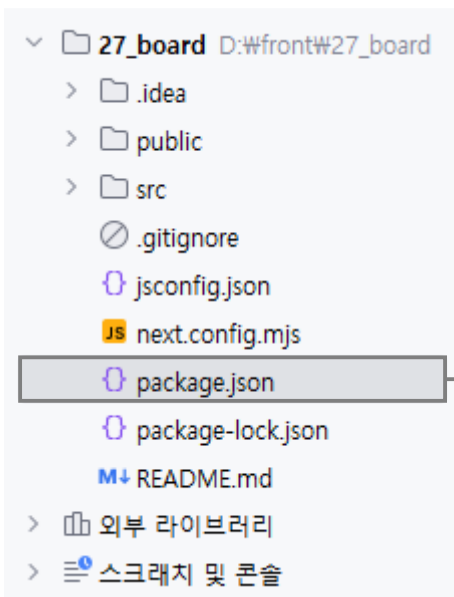
빌드

- 특별한 설정이 없다면 기본 포트는 3000 번으로 실행

▲ Next.js 15.5.4 (Turbopack)

- Local: <http://localhost:3000>
- Network: <http://192.168.0.13:3000>

- 만약 포트를 변경하고 싶다면 package.json 에 아래 내용을 추가해서 변경 가능



```
{
  "name": "27_board",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev --turbo",
    "build": "next build --turbo",
    "start": "next start -p 5000"
  },
}
```



▲ Next.js 15.5.4 (Turbopack)

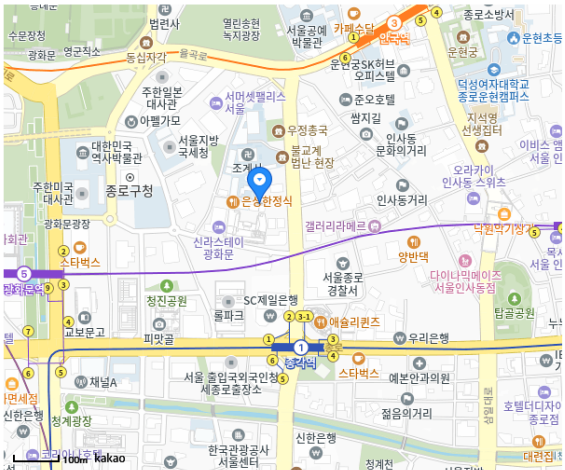
- Local: <http://localhost:5000>
- Network: <http://192.168.0.13:5000>

Summary

외부라이브러리

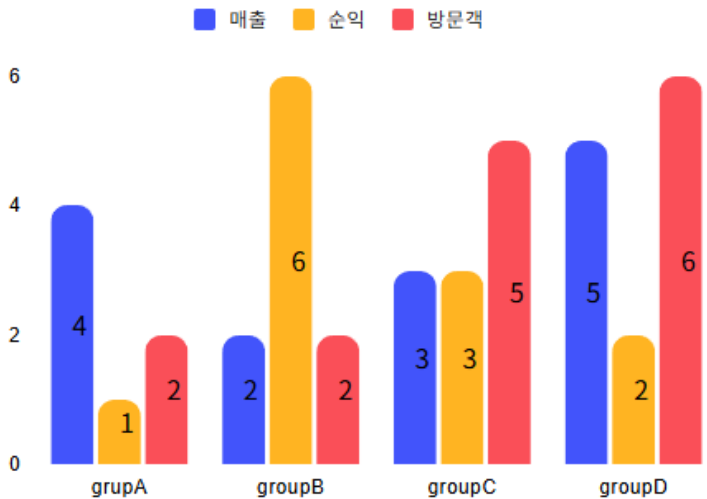
카카오 맵	맵 서비스 활용
chart.js	그래프를 보다 손쉽게 생성
Boot Strap	세련된 UI를 보다 손쉽게 생성
pagination	페이징 UI 를 보다 손쉽게 생성

- 카카오 맵



클릭한 위치의 위도는 37.5730236332742, 경도는 126.98215968778834 입니다.

- Chart.js



- Bootstrap



- Pagination

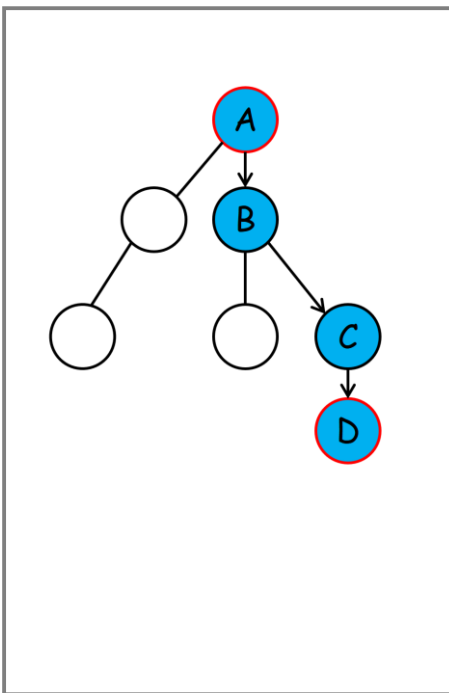


Summary

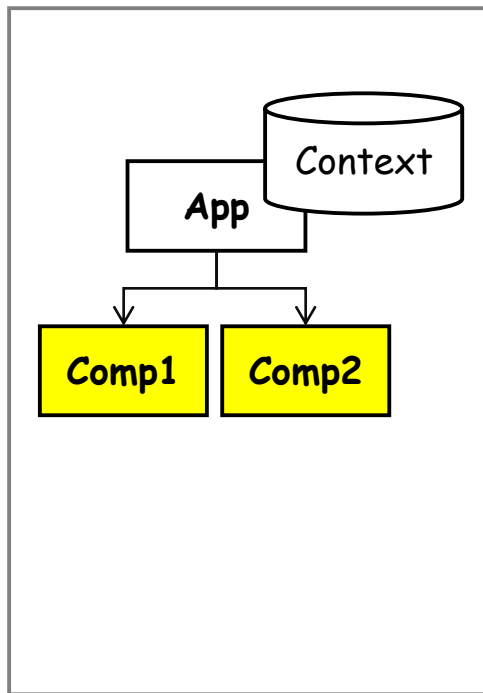
☑ 데이터 공유 방식

- Context는 생성한 데이터를 모든 자식 컴포넌트에서 받아볼 수 있는 저장소 개념
- 다만 Context 값 변경 시 Context 를 공유하는 모든 Component 가 렌더링 됨
- Redux를 활용하면 Context 사용 시 보다 효율적인 상태 관리가 가능

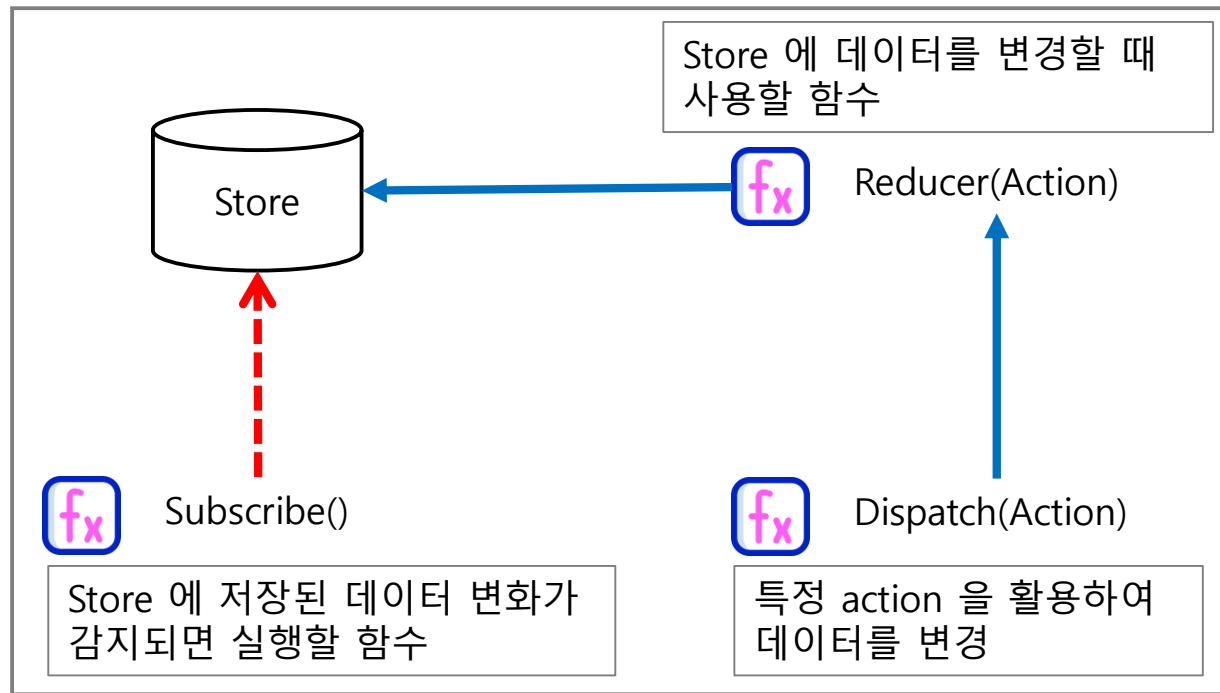
• props



• context



• Redux



Summary



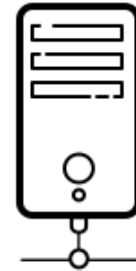
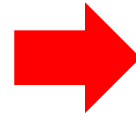
- 빌드(Build)란 개발한 소스 코드를 사용어나 다른 시스템에서 실행 가능한 형태로 변환하는 과정
- 배포(deploy)는 빌드 단계에서 생성된 산출물을 서버에 설치하고 작동시키는 과정



source



build



deploy

Chapter Summary

☑ React.js & Next.js

- Next.js 는 React.js 에서의 단점을 개선한 react.js 의 framework
- Next.js 는 폴더구조로써 Routing 설정이 가능

☑ JWT

- JSON Web Token 은 서버와의 통신을 위해 사용

header

.

payload

.

signature

☑ 상태 공유

- React.js 의 상태는 부모로 부터 자식에게 일방향으로만 전달 가능(prop drilling)
- Context 를 통해 해결할 수 있지만 Context 공유 객체들의 동반 렌더링 문제가 있음
- 이를 해결하기 위해 Redux 등의 상태 공유 Framework들이 있음

과정 Summary

Virtual DOM

- React.js 는 화면 변경 시 virtual DOM 을 활용
- UI 의 작은 변화가 있는 곳에 효율적

Component

- React.js 는 Component 를 조합하여 UI를 구성
- 각 Component 에는 변경 시 UI 가 rendering 되는 props 와 state 객체를 사용

Routing

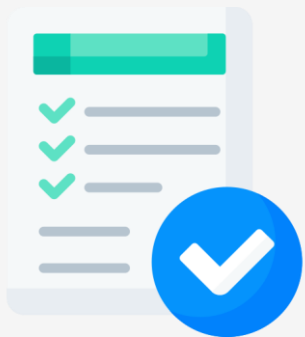
- SPA 에 특화된 React.js 는 페이지 이동이 많은 경우 Router 를 사용
- Next.js 는 폴더 구조로 Router 를 사용할 수 있음

Build

- 개발이 완성된 소스는 Build 를 통해 최종 서버에 배포되어야 함

과정 Summary

수고하셨습니다!



이제 여러분은 다음과 같은 내용을 수행할 수 있습니다.

1. Component 를 활용한 UI 를 구성 할 수 있다.
2. State와 props를 활용해 상태값 변화에 따른 UI 변화를 줄 수 있다.
3. Next.js를 활용해 Routing 구조와 페이지를 생성할 수 있다.
4. 프로젝트를 위한 외부 라이브러리를 활용할 수 있다.
5. 서버와의 통신을 통해 기능을 구현할 수 있다.
6. 상태 공유 Framework를 활용할 수 있다.
7. 완성된 프로젝트를 build 하여 배포할 수 있다.